

半平面交

注意一下：半平面有交 $\neq$ 半平面交面积大于零，半平面交可以是一点或者一条线段，但是面积为零

```
1 // 半平面交：传入一组有向直线，求解它们左侧交集构成的凸多边形，O(n log n)
2 vector<Line> halfinter(vector<Line> l, const T lim = 1e9) {
3     const auto check = [] (const Line &a, const Line &b, const Line &c) {
4 #ifdef usedouble
5         return a.toleft(b.inter(c)) < 0;
6 #else
7         // 无精度误差的方法，但注意取值范围会扩大到三次方
8         /* 原版如下，改了一版 __int128 展开，但没测展开对不对，慎用
9             const Point p=a.v*(b.v^c.v),q=b.p*(b.v^c.v)+b.v*(c.v^(b.p-c.p))-
10             a.p*(b.v^c.v);
11             return p.toleft(q)<0;
12         */
13         T Sbc = c.v ^ (b.p - c.p);
14         T Xbc = b.v ^ c.v;
15         __int128_t sum = (__int128_t)Xbc * (a.v ^ (b.p - a.p)) +
16         (__int128_t)Sbc * (a.v ^ b.v);
17         if(Xbc > 0) return sum < 0;
18         if(Xbc < 0) return sum > 0;
19         return false;
20 #endif
21     };
22     // 添加一个大边界框 lim 限制范围，防止无界半平面
23     l.push_back({{-lim, 0}, {0, -1}}); l.push_back({{0, -lim}, {1, 0}});
24     l.push_back({{lim, 0}, {0, 1}}); l.push_back({{0, lim}, {-1, 0}});
25     sort(l.begin(), l.end());
26     deque<Line> q;
27     for(size_t i = 0; i < l.size(); i++) {
28         if(i > 0 && l[i - 1].v.toleft(l[i].v) == 0 && l[i - 1].v * l[i].v >
29         eps) continue;
30         while(q.size() > 1 && check(l[i], q.back(), q[q.size() - 2]))
31             q.pop_back();
32         while(q.size() > 1 && check(l[i], q[0], q[1])) q.pop_front();
33         if(!q.empty() && q.back().v.toleft(l[i].v) <= 0) return vector<Line>();
34         // 交集为空
35         q.push_back(l[i]);
36     }
37     while(q.size() > 1 && check(q[0], q.back(), q[q.size() - 2])) q.pop_back();
38     while(q.size() > 1 && check(q.back(), q[0], q[1])) q.pop_front();
39     return vector<Line>(q.begin(), q.end()); // 交集非空，但交集面积可能为零
40 }
```

